

Lexicon Lookup Engine

Submitted By

Student Name	Student ID
Nabila Mehzabin	253-15-162
Bayezid Bostami	253-15-442
Humayra Mehzabin	253-15-163
Sanjidur Rahman	253-15-662
Md. Mahinur Rahman Khan	253-15-151

MINI LAB PROJECT REPORT

This Report is presented in Partial Fulfillment of the course

CSE124: Data Structure Lab
in the Computer Science and Engineering Department



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

04/08/2026

DECLARATION

We hereby declare that this lab project was done by us under the supervision of Jamilul Huq Jami, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as a lab project.

Submitted To:

Jamilul Huq Jami

Lecturer

Department of Computer Science and Engineering

Daffodil International University

Submitted By:

_____ Md. Mahinur Rahman Khan ID: 253-15-151	
_____ Nabila Mehzabin ID: 253-15-162	_____ Humayra Mehzabin ID: 253-15-163
_____ Bayezid Bostami ID: 253-15-442	_____ Sanjidur Rahman ID: 253-15-662

COURSE & PROGRAM OUTCOME

The following courses have course outcomes as follows:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Define and relate classes, objects, members of the class, and relationships among them needed for solving specific problems.
CO2	Formulate knowledge of programming and C in problem-solving.
CO3	Analyze Data structures to present a specific problem.

Table 2: Mapping of CO, PO, Blooms, KP, and CEP

CO	PO	Blooms	KP	CEP	CEA
CO1	PO1	C1, C2, P1, P2, P3	K3	EP1	
CO2	PO2	C2, C3, P1, P2, A2	K3, K4	EP2	
CO3	PO3	C3, C6, P1, P2, P3, P4, A1, A2, A3	K5	EP3	

The mapping justification of this table is provided in the Engineering Standards and Mapping sections of this report.

Contents

Chapter 1:

Introduction

1.1 Introduction	5
1.2 Motivation	5
1.3 Objectives	6
1.4 Feasibility Study	6
1.5 Gap Analysis	7
1.6 Project Outcome.	7

Chapter 2:

Proposed Methodology/Architecture

2.1 Requirement Analysis & Design Specification	9
2.2 Design Specification.	9
2.3 System Architecture Overview	10
2.4 Authentication & Access Control Module.	10
2.5 UI Design	11 & 12

Chapter 3:

Implementation and Results

3.1 Core Implementations	13
3.1.1 Data Structures and Global Definitions.	13 & 14
3.1.2 Queue Logic (Search History)	15
3.1.3 AVL Tree Rotations and Balancing.	16
3.1.4 Tree Insertion Logic	17
3.1.5 Stack Undo Mechanics	18
3.2 Performance Analysis.	19
3.2.1 Time Complexity.	19
3.2.2 Memory Usage.	19

3.3 Results and Discussion	20
3.3.1 Execution Trace Example.	20

Chapter 4:

Engineering Standards and Mapping

4.1 Impact on Society, Environment and Sustainability.	21
4.1.1 Impact on Practical Skills	21
4.1.2 Impact on Society & Environment.	21
4.1.3 Ethical Aspects.	21
4.2 Complex Engineering Problem (CEP)	22
4.3 Mapping of Program Outcome	22
4.3.1 Rationale for Complex Problem Solving	22

Chapter 5:

Conclusion

5.1 Summary	23
5.2 Limitation.	23
5.3 Future Work.	24

Chapter 6:

Reference.	23 & 24
--------------------	---------

Chapter 1

Introduction

This chapter provides an overview of the Lexicon Lookup Engine project, outlining the background, motivation, objectives, feasibility, gap analysis, and expected outcomes of the developed software solution.

1.1 Introduction

A Lexicon Lookup Engine is a highly optimized, console-based software solution designed to simulate the core functionality of a modern dictionary application. The system organizes an extensive vocabulary of words and their corresponding definitions, allowing the user to search, insert, and delete entries dynamically. Unlike standard linear data applications, this engine relies on advanced tree-based data structures to achieve rapid lookup times, while concurrently maintaining a history of searches and providing robust undo capabilities.

The application is entirely implemented in the C Programming Language. The engine demonstrates how complex, real-world application features—such as instantaneous search results, action rollbacks, and persistent history—can be constructed purely from fundamental algorithmic components. By integrating a Binary Search Tree (BST), an Adelson-Velsky and Landis (AVL) Tree, a Stack, a Queue, and a Singly Linked List, this project serves as a comprehensive case study in applied data structures.

1.2 Motivation

The primary motivation behind this project stems from the universal reliance on fast information retrieval systems. Every computer user interacts with search engines, spell-checkers, and digital dictionaries daily. The underlying logic that makes these systems responsive, even with vast datasets, depends entirely on how the data is stored and traversed in memory.

We wanted to move beyond the theoretical study of nodes and pointers and build an application where the performance difference between data structures is measurable and explicit. Implementing an AVL tree alongside a standard BST allows us to visibly benchmark the efficiency of self-balancing trees. Furthermore, the inclusion of modern application features—like an "Undo" stack (similar to Ctrl+Z in word processors) and a "Recent Searches" queue (similar to browser history)—motivated us to design a system that feels like a complete, professional application rather than a disjointed collection of algorithms.

1.3 Objectives

The core objectives of the Lexicon Lookup Engine are strictly defined to ensure comprehensive coverage of the Data Structures curriculum:

- **Efficient Search and Retrieval:** To design a searchable lexicon using both a standard Binary Search Tree (BST) and a self-balancing AVL Tree to provide logarithmic $O(\log n)$ lookup times, and to compare their comparison metrics in real-time.
- **Action Reversibility:** To implement an “Undo” feature using a Stack (LIFO structure), allowing users to safely roll back accidental insertions or deletions in $O(1)$ time complexity.
- **Session Tracking:** To build a limited capacity “Recent Searches” tracker using a Queue (FIFO structure) that automatically overwrites the oldest searches when the capacity is reached.
- **Persistent Storage:** To maintain a Singly Linked List as the master memory bank, which synchronizes with a flat-file database (`lexicon_db.txt`), ensuring data is not lost between program executions.
- **Unified Interface:** To create a user-friendly, menu-driven Command Line Interface (CLI) that seamlessly orchestrates these distinct data structures.

1.4 Feasibility Study

A feasibility study is an assessment of the practicality of a proposed project, analyzing whether it can be successfully implemented using available resources.

- **Technical Feasibility:** Search and dictionary systems are well-established domains. Simplified versions are universally implemented to teach algorithmic complexity. Developing a tree-based dictionary in C using standard standard library functions (malloc, free, file I/O) is completely viable within the scope of a semester lab project.
- **Resource Constraints:** The application requires minimal computational overhead. Because it runs purely in terminal memory (RAM) and writes to a simple .txt file, it will execute smoothly on any modern operating system equipped with a C compiler (e.g., GCC, Clang) without requiring third-party libraries or external database servers.

1.5 Gap Analysis

Gap analysis compares existing solutions with potential or desired performance to identify areas for improvement or educational value.

- **Commercial Complexity vs. Educational Clarity:** Real-world applications like dictionary.com rely on massive relational databases (SQL) or NoSQL document stores, caching layers (Redis), and cloud infrastructure. These abstract away the data structures.

Our gap analysis indicates that an educational project should make the underlying structures explicit.

- **Algorithmic Transparency:** By outputting the exact number of comparisons made by the BST versus the AVL tree during a search, this engine fills the gap between theoretical Big-O notation and practical application, providing immediate visual feedback on algorithmic efficiency.

1.6 Project Outcome

The expected outcomes of this project are:

1. A fully functional, persistently stored Lexicon Lookup Engine written in C.
2. A robust demonstration of five distinct data structures operating simultaneously without memory leaks.
3. Fast, dynamically balanced $O(\log n)$ dictionary lookups preventing worst-case linear time degradation.
4. An operational Stack-based Undo mechanism capable of reversing database mutations.
5. A working Queue-based history tracker demonstrating ring-buffer style eviction.

Chapter 2

Proposed Methodology/Architecture

This chapter details the methodology used to engineer the Lexicon Lookup Engine, including the requirement analysis, architectural specifications, and the UI layout.

2.1 Requirement Analysis & Design Specification

The Requirement Analysis phase focuses on identifying the constraints and needs of the software. The system was designed around the behavior of a professional CLI application.

2.1.1 Functional Requirements

- **Lexicon Management:** The system must allow words and definitions to be inserted, deleted, and searched.

- **Search History Logging:** Every successful or unsuccessful search attempt must be logged to a recent history list limited to the last 10 entries.
- **Undo Mechanism:** Deletions and insertions must be recorded as inverse operations so that a user can revert the last modification.
- **Data Persistence:** The engine must read from a database file on startup and write changes back to it during mutations.
- **Case Insensitivity:** The system must convert all inputs to lowercase to ensure searches are completely case-insensitive.

2.1.2 Non-Functional Requirements

- **Scalability (Logarithmic Time):** The search speed must remain fast even if thousands of words are loaded. A linear scan ($O(n)$) is unacceptable.
- **Memory Safety:** Removed nodes must be explicitly deallocated using 'free()' to prevent memory leaks during long sessions.
- **Usability:** A clean, numerical CLI interface must guide the user smoothly through operations.

2.2 Design Specification

This project deliberately layers multiple data structures. Rather than storing multiple copies of strings, the trees hold pointers to the master Singly Linked List nodes.

2.2.1 Data Structure Design

- **Master Storage (Singly Linked List):** The WordNode struct forms a linked list that holds the actual character arrays for the word and definition. This acts as the central repository.
- **Fast Lookup (AVL Tree & BST):** We run a standard Binary Search Tree and a selfbalancing AVL Tree in parallel. When a node is added to the Master list, a pointer to that node is inserted into both trees based on alphabetical string comparison (strcmp). The AVL tree mathematically guarantees an $O(\log n)$ height by performing Left and Right rotations whenever the balance factor exceeds 1 or falls below -1.
- **Undo Operations (Stack):** The StackNode forms a Last-In-First-Out (LIFO) structure. When a word is inserted, an ACTION_INSERT flag (along with the word payload) is pushed to the stack. If "Undo" is triggered, the stack is popped, and the engine executes the opposite command (deletion).

- **Search Tracking (Queue):** The Queue structure operates on a First-In-FirstOut (FIFO) basis. When the search capacity reaches 10, the oldest search at the front pointer is dequeued, and the newest search is enqueued at the rear.

2.2.2 Module Design

The codebase is heavily modularized for debugging and maintainability.

- **Tree Rotations:** `rightRotate()` and `leftRotate()` handle the intricate pointer manipulation required to balance the AVL tree.
- **Core Operations:** `insertLexiconEntry()`, `deleteLexiconEntry()`, and `searchWordCommand()` orchestrate the updating of the Linked List, the trees, and the file system simultaneously.
- **History/Undo Modules:** `enqueueHistory()`, `dequeueHistory()`, `pushUndoAction()`, and `processUndo()` encapsulate the logic for the auxiliary data structures.

2.3 System Architecture Overview

The software flows linearly from initialization to an infinite event loop.

1. **Boot:** The program allocates the Queue.
2. **Load:** `loadDatabase()` reads `lexicon_db.txt` line by line, parsing words and definitions, and feeding them into the Master List, BST, and AVL Tree.
3. **Event Loop:** `main()` presents the CLI menu.
4. **Dispatch:** User input (1-6) routes the program to the corresponding core operation.
5. **Teardown:** On exit (Option 6), the program terminates. (Note: OS reclaims memory, but persistent changes are already synced to disk).

2.4 Authentication & Access Control Module

The system incorporates a robust authentication layer to manage user access and restrict entry to the core Lexicon Engine. This ensures that only verified users can interact with the data structures and persistent files.

2.4.1 Functional Requirements for Authentication

- **User Registration & Login:** The system must allow new users to register by providing a unique username and password and permit existing users to authenticate their credentials.
- **Duplicate Prevention:** During registration, the system must perform a scan of the existing database to ensure the requested username is not already taken.
- **State-Driven Session Management:** The primary event loop must be controlled by an authentication flag (isLoggedIn). Users are confined to a distinct Authentication Menu until verified.
- **Lazy Loading (Memory Optimization):** The core lexicon data structures (AVL Tree, BST, Master List) and the lexicon_db.txt file are purposefully not booted or loaded into memory until the first successful login occurs.

2.4.2 Module Design & Persistence

- **Credential Storage (File I/O):** User credentials are persistently stored in a dedicated flatfile database (users_db.txt). Data is appended and parsed using a pipe-delimited string format (username|password).
- **Authentication Functions:** The codebase encapsulates authentication logic into specific modules. registerUser() handles input gathering and database appending, loginUser() parses the database for exact string matches using strcmp(), and isUsernameTaken() acts as a validation helper.
- **UI State Separation:** The Command Line Interface is explicitly split into two states. The application relies on state-checking to switch the terminal UI between the login screen and the main Lexicon App screen, ensuring secure logical flow.

2.5 UI Design

Login page UI.

```
=====
WELCOME TO THE LEXICON APP
=====
1. Register
2. Log In
3. Exit
Enter your choice:
```

Post Login page UI.

```
=====

##      #####  ## ## ##### ##### ##### ##  ##
##      ##      ## ##   ##   ##      ##   ##   ##
##      #####   ##   ##   ##      ##   ##   ##
##      ##      ## ##   ##   ##      ##   ##   ##
##      ##      ## ##   ##   ##      ##   ##   ##
##### #####  ##  ## ##### ##### ##### ##  ##

      #####  #####  #####
      ## ## ## ## ## ##
      #####  #####  #####
      ## ## ##   ##
      ## ## ##   ##

=====
```

1. Search for a Word
 2. Insert a New Word
 3. Delete a Word
 4. View Recent Search History (Queue)
 5. Undo Last Action (Stack)
 6. Log Out
 7. Exit Application
- ```
=====
```

# **Chapter 3**

## **Implementation and Results**

This chapter details the exact C implementation of the Lexicon Lookup Engine, providing the source code blocks, mathematical analysis of the structures, and execution results.

### **3.1 Core Implementations**

#### **3.1.1 Data Structures and Global Definitions**

The foundational structures define the nodes for our linked list, trees, stack, and queue.

```

#define MAX_WORD_LEN 100
#define MAX_DEF_LEN 2000
#define DB_FILE "lexicon_db.txt"

// Singly Linked List (Master Storage)
typedef struct WordNode { char
word[MAX_WORD_LEN]; char
definition[MAX_DEF_LEN]; struct
WordNode* next; } WordNode;

// Standard BST Node
typedef struct BSTNode {
WordNode* data; struct
BSTNode* left; struct
BSTNode* right; }
BSTNode;

// AVL Tree Node typedef
struct AVLNode {

```

```

1 2 3 4 5 6 7 8
9 10
11 12 13
14
15
16
17 18 19 20

```

```

 WordNode* data; struct
 AVLNode* left; struct
 AVLNode* right; int
 height;
} AVLNode;

// Stack Node (Undo operations)
typedef enum { ACTION_INSERT, ACTION_DELETE } ActionType;

typedef struct StackNode {
 ActionType action;
 char word[MAX_WORD_LEN];
 char definition[MAX_DEF_LEN];
 struct StackNode* next; }
StackNode;

// Queue Node & Queue (Recent Searches)
typedef struct QueueNode { char
searchWord[MAX_WORD_LEN]; struct
QueueNode* next;
} QueueNode;

typedef struct Queue {
 QueueNode* front;
 QueueNode* rear; int
 count;
} Queue;

// Globals
WordNode* masterListHead = NULL;
BSTNode* bstRoot = NULL;
AVLNode* avlRoot = NULL;
StackNode* undoStackTop = NULL;
Queue* searchHistory = NULL;

```

```

21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
37
38 39 40 41 42 43 44 45 46 47 48
49
50 51 52 53
54

```

Listing 3.1: Struct Definitions and Globals

**Analysis:** Notice that BSTNode and AVLNode do not duplicate the word strings. They merely contain a pointer (WordNode\* data) to the master list. This heavily optimizes memory usage. The Stack utilizes an enum to remember whether the past action was an insertion or deletion.

### 3.1.2 Queue Logic (Search History)

The queue operates using a front and rear pointer to maintain  $O(1)$  enqueue and dequeue times.

```
Queue* createQueue(void) {
 Queue* q =
 (Queue*)malloc(sizeof(Queue)); q->front =
 NULL; q->rear = NULL; q->count = 0;
 return q;
}

void dequeueHistory(void) {
 if (searchHistory->front == NULL) return;
 QueueNode* temp = searchHistory->front;
 searchHistory->front = searchHistory->front->next;
 if (searchHistory->front == NULL) searchHistory->rear = NULL; free(temp);
 searchHistory->count--;
}

void enqueueHistory(const char* word) {

 if (searchHistory == NULL) searchHistory = createQueue();

 if (searchHistory->count == MAX_HISTORY) dequeueHistory();

 QueueNode* newNode = (QueueNode*)malloc(sizeof(QueueNode));

 strcpy(newNode->searchWord, word);

 newNode->next = NULL; if (searchHistory->rear ==
 NULL) { searchHistory->front = searchHistory->rear =
 newNode;
 } else { searchHistory->rear->next =
 newNode; searchHistory->rear =
 newNode;
 } searchHistory->count++;
}
```

```
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16
17
18 19 20 21 22 23 24 25 26 27 28
29
30 31 32
33
```

Listing 3.2: Queue Operations

**Analysis:** The enqueueHistory function ensures the queue never exceeds MAX\_HISTORY (set to 10). If it is full, it immediately calls dequeueHistory() before appending the new node to the rear.



### 3.1.3 AVL Tree Rotations and Balancing

The core complexity of this project lies in the AVL tree's self-balancing mechanism.

```
int getHeight(AVLNode* n) { return
n ? n->height : 0;
}
int max(int a, int b) {
return (a > b) ? a : b;
}
int getBalance(AVLNode* n) {
return n ? getHeight(n->left) - getHeight(n->right) : 0;
}

AVLNode* rightRotate(AVLNode* y) {
 AVLNode* x = y->left;
 AVLNode* T2 = x->right; x->right
 = y; y->left = T2;
 y->height = max(getHeight(y->left), getHeight(y->right)) + 1; x-
 >height = max(getHeight(x->left), getHeight(x->right)) + 1; return x;
}

AVLNode* leftRotate(AVLNode* x) {
 AVLNode* y = x->right; AVLNode*
 T2 = y->left;
 y->left = x; x->right
 = T2;
 x->height = max(getHeight(x->left), getHeight(x->right)) + 1; y-
 >height = max(getHeight(y->left), getHeight(y->right)) + 1; return
 y;
}
```

Listing 3.3: AVL Tree Rotations

**Analysis:** Whenever an insertion or deletion unbalances the tree (Balance Factor  $> 1$  or  $< -1$ ), these rotation functions adjust the parent-child pointers to restore a strict  $O(\log n)$  height while preserving the binary search property (left children are smaller, right children are larger).

### 3.1.4 Tree Insertion Logic

```
AVLNode* insertAVL(AVLNode* node, WordNode* data) {

 if (!node)

 return createAVLNode(data);

 int cmp = strcmp(data->word, node->data->word);

 // Normal BST insertion
 if (cmp < 0) node->left = insertAVL(node->left, data);

 else if (cmp > 0) node->right = insertAVL(node->right, data);

 else return node;

 // Update height and check balance

 node->height = 1 + max(getHeight(node->left), getHeight(node->right));

 int balance = getBalance(node);

 // Left Left Case if (balance > 1 && strcmp(data->word, node->left-
 >data->word) < 0) return rightRotate(node);

 // Right Right Case
 if
 (balance < -1 && strcmp(data->word, node->right->data->word) > 0) return
 leftRotate(node);
 // Left Right Case if (balance > 1 && strcmp(data->word, node->left-
 >data->word) > 0) { node->left = leftRotate(node->left); return
 rightRotate(node);
 }
 // Right Left Case
 if (balance < -1 && strcmp(data->word, node->right->data->word) < 0) {
 node->right = rightRotate(node->right); return leftRotate(node);
 } return
 node;
}
```

1 2 3  
4  
5  
6  
7 8  
9 10  
11  
12 13 14 15

16 17

18 19 20 21 22 23 24 25 26 27 28

## Listing 3.4: AVL Insertion

## 3.1.5 Stack Undo Mechanics

```

void processUndo(void) { if
(undoStackTop == NULL) {
printf("\n[Undo] Nothing to undo!\n");
return;
}
StackNode* lastAction = undoStackTop; undoStackTop
= undoStackTop->next;

if (lastAction->action == ACTION_INSERT) {
printf("\n[Undo] Undoing Insertion. Deleting '%s'...\n", lastAction->word);
deleteLexiconEntry(lastAction->word, 1);
} else if (lastAction->action == ACTION_DELETE) { printf("\n[Undo]
Undoing Deletion. Restoring '%s'...\n", lastAction
->word);
insertLexiconEntry(lastAction->word, lastAction->definition, 1); appendToDatabase(lastAction-
>word, lastAction->definition);
} free(lastAction);
}

```

1 2 3 4 5 6 7 8

9 10

11 12

13

14

15 16

17

18

## Listing 3.5: Stack Operations for Undo

**Analysis:** processUndo operates in  $O(1)$  time to retrieve the top of the stack. It then executes the inverse of the logged action, safely reversing user mistakes.

## 3.2 Performance Analysis

A critical requirement of this project was to analyze the time and space complexity of the system as the database scales.

### 3.2.1 Time Complexity

- **Search Operation (searchAVL):** The AVL tree guarantees a maximum depth of approximately  $1.44 \times \log_2(n)$ . Searching for a word in a dictionary of 1,000,000 entries requires at most 20 comparisons. A standard BST could degrade into a linked list in the worst case (e.g., words added in alphabetical order), resulting in 1,000,000 comparisons ( $O(n)$ ).
- **Insertion Operation (insertAVL):** Insertion takes  $O(\log n)$  to find the correct leaf node, followed by at most two rotations ( $O(1)$  time) to rebalance the tree.
- **Undo Operation (processUndo):** Popping from the stack is  $O(1)$ . Applying the inverse action (insert/delete) involves tree traversal, costing  $O(\log n)$ .
- **History Operation (enqueueHistory):** Queue updates involve updating front/rear pointers, strictly taking  $O(1)$  constant time.

### 3.2.2 Memory Usage

Because the trees hold pointers to the WordNode rather than character arrays, memory is strictly conserved. Each entry requires:

- 1 WordNode:  $100 + 2000 + 8$  (pointer) = 2108 bytes.
- 1 BSTNode:  $8$  (data) +  $8$  (left) +  $8$  (right) = 24 bytes.
- 1 AVLNode:  $8$  (data) +  $8$  (left) +  $8$  (right) +  $4$  (int height) = 28 bytes.

Total per word = 2160 bytes. Even a massive library of 100,000 words consumes merely  $\sim 216$  MB of RAM, well within modern system capabilities.

## 3.3 Results and Discussion

The implemented Lexicon Lookup Engine successfully achieves all functional parameters.

### 3.3.1 Execution Trace Example

#### Scenario 1: Insertion and Search Benchmarking

Enter your choice: 2

Enter new word: algorithm

Enter definition: A step-by-step procedure for solving a problem.

[Success] 'algorithm' added and saved.

Enter your choice: 1

Enter word to search: algorithm

[Found] algorithm: A step-by-step procedure for solving a problem. [Benchmark]

AVL Comparisons: 2 | BST Comparisons: 2

### Scenario 2: The Undo Feature

Enter your choice: 3 Enter word to delete:

algorithm

[Success] 'algorithm' deleted.

Enter your choice: 5

[Undo] Undoing Deletion. Restoring 'algorithm'...

[Success] 'algorithm' added and saved.

**Discussion:** The dual-tree search demonstrates theoretical parity when small amounts of randomized data are inserted. However, if the database is loaded from an alphabetically sorted flat file, the BST comparisons skyrocket while the AVL comparisons remain rigidly logarithmic. This actively proves the necessity of self-balancing algorithms in commercial database environments. The Stack and Queue perform flawlessly, offering quality-of-life features expected in modern graphical applications, entirely via terminal output.

## Chapter 4

# Engineering Standards and Mapping

This chapter details how the project aligns with program outcomes, engineering problem solving, and global sustainability standards.

### 4.1 Impact on Society, Environment and Sustainability

The Lexicon Lookup Engine addresses software efficiency, reflecting resource-conscious engineering.

#### 4.1.1 Impact on Practical Skills

This lab project reinforces the transition from abstract algorithmic theory to applied software development. Understanding that standard commercial features (Undo mechanisms, Browser

History, Database Indexing) rely purely on foundational Data Structures (Stacks, Queues, Trees) grounds the curriculum in practical reality.

#### 4.1.2 Impact on Society & Environment

By deliberately optimizing the architecture (e.g., utilizing pointers to master data rather than duplicated strings, using AVL trees to drastically reduce CPU cycles during lookup), the engine reduces computational overhead. Extrapolated to serverscale applications, minimizing time complexity directly reduces electrical consumption and cooling requirements in data centers, contributing to Green IT initiatives.

#### 4.1.3 Ethical Aspects

Data manipulation systems must prioritize data integrity. The engine ensures that no database writes are executed destructively without the option to revert (processUndo). Furthermore, string sanitization (toLowerCase) ensures uniform accessibility, preventing users from creating duplicate, conflicting entries differing only by case.

### 4.2 Complex Engineering Problem (CEP)

The development of a multi-structure synchronized engine constitutes a Complex Engineering Problem.

- **Synchronization Context:** Deleting a word means the BSTNode, the AVLNode, and the persistent .txt file must all be updated safely without invalidating pointers or causing segmentation faults.
- **Constraint Handling:** Writing an AVL self-balancing routine requires managing deeply interdependent variables (left/right heights and balance factors). An error in rotation logic breaks the binary search property, rendering the dictionary useless.

### 4.3 Mapping of Program Outcome

**Table: Justification of Program Outcomes**

| PO's | Justification                                                                                                                                                            |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PO1  | <b>Engineering Knowledge:</b> Applied advanced discrete mathematics and algorithmic principles to construct a self-balancing AVL tree and memoryefficient node pointers. |

|     |                                                                                                                                                                                                                      |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PO2 | <b>Problem Analysis:</b> Identified the degradation problem inherent to standard Binary Search Trees when provided sorted data, and formulated an AVL-based solution to ensure consistent $O(\log n)$ performance.   |
| PO3 | <b>Design/Development of Solutions:</b> Designed a robust, scalable command-line application that seamlessly integrates five unique data structures while ensuring memory safety and persistent file I/O operations. |

### 4.3.1 Rationale for Complex Problem Solving

- **EP1 (Depth of Knowledge):** The system requires understanding pointer arithmetic, memory allocation, string sanitization, and rotational matrix logic for trees.
- **EP2 (Conflicting Requirements):** The conflict between memory conservation and fast lookup was solved by utilizing a single Master Linked List, allowing the BST and AVL trees to merely store lightweight referencing pointers.
- **EP3 (Depth of Analysis):** Implementing a benchmark counter inside the search functions allows analytical evaluation of tree depth degradation under various data loads.

## Chapter 5

## Conclusion

This final chapter summarizes the project's achievements, acknowledges its current limitations, and outlines potential avenues for future enhancements.

### 5.1 Summary

The Lexicon Lookup Engine successfully demonstrates the practical implementation and orchestration of advanced data structures in C. By combining a Singly Linked List for master memory storage, an AVL Tree for guaranteed logarithmic lookups, a standard Binary Search Tree for performance benchmarking, a Stack for action reversibility, and a Queue for session history, the project models the backend architecture of professional dictionary applications. The command-line interface provides an intuitive portal for users to insert, search, delete, and undo operations safely. Performance analyses prove that the AVL tree effectively prevents the linear time degradation that plagues standard BSTs during sorted insertions, meeting the primary technical objectives of the project. Data persistence via a flat text file ensures the engine is a usable, long-term utility rather than a temporary memory exercise.

## 5.2 Limitation

Despite its successes, the current implementation possesses several technical limitations:

- **Deletion Memory Management:** While the tree nodes are freed upon deletion, removing the underlying string payload from the Singly Linked List (Master Storage) is currently deferred. Over a massive number of insertions and deletions, this results in orphaned data in the master list, consuming excess RAM.
- **Database Sync Overhead:** The `syncDatabaseToFile()` function rewrites the entire lexicon to the `.txt` file upon every deletion. For exceptionally large databases (e.g., millions of entries), this  $O(n)$  disk write operation will introduce noticeable latency.
- **Definition Constraints:** The definitions are limited to a hardcoded `MAX_DEF_LEN` of 2000 characters. Exceeding this via input causes buffer truncation.

## 5.3 Future Work

To elevate the engine to commercial grade, several enhancements are proposed:

- **Trie Structure Implementation:** Replacing the AVL tree with a Prefix Tree (Trie) would enable advanced features like autocomplete and wildcard searching, which are standard in modern search engines.
- **Dynamic String Allocation:** Implementing dynamic string sizing instead of fixed-length arrays will completely optimize memory utilization and remove the 2000-character constraint on definitions.
- **Database Hashing / Chunking:** Updating the file I/O logic to write to randomaccess binary files or utilizing a hash map for disk storage would eliminate the  $O(n)$  rewrite penalty during deletions.

## References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [2] Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.).



Prentice Hall.

- [3] Department of Computer Science & Engineering. *Data Structures Course Slides*. Daffodil International University BLC (Summer 2026).
- [4] GeeksforGeeks. (n.d.). *Data Structures: Linked List, Stack, Queue, AVL Tree, and Binary Search Tree (BST)*. Retrieved from <https://www.geeksforgeeks.org/datastructures/>